YAĞMUR ÖZTEKİN

MİGRATİONS

Migrations, Django'nun modellerinizde yaptığınız değişiklikleri (alan ekleme, model silme vb.) veritabanı şemanıza yayma yöntemidir. Çoğunlukla otomatik olacak şekilde tasarlanmışlardır, ancak migrasyonları ne zaman oluşturmanız, ne zaman çalıştırmanız gerektiğini ve karşılaşabileceğiniz yaygın sorunları bilmeniz gerekir.

Komutlar

Veritabanı şemalarının Django tarafından işlenmesi ve migration'larla etkileşim kurmak için kullanacağınız birkaç komut vardır:

Migrate=Göç işlemlerini uygulamaktan ve geri almaktan sorumlu olan birim.

Makemigrations=Bu, modellerinizde yaptığınız değişikliklere göre yeni geçişler oluşturmaktan sorumludur.

Sqlmigrate=Bu, bir geçiş işlemi için SQL ifadelerini görüntüler.

Showmigrations=Bu, bir projenin geçişlerini ve durumlarını listeler.

Veritabanı şemanız için bir sürüm kontrol sistemi olarak geçişleri düşünmelisiniz. `migration`, **makemigrations** model değişikliklerinizi ayrı ayrı geçiş dosyalarına paketlemekten (tıpkı commit'ler gibi) ve **migrate** bu değişiklikleri veritabanınıza uygulamaktan sorumludur.

Her uygulama için geçiş dosyaları, o uygulamanın içindeki "migrations"dizininde bulunur ve kod tabanının bir parçası olarak işlenip dağıtılmak üzere tasarlanmıştır. Bu dosyaları bir kez geliştirme makinenizde oluşturmalı ve ardından aynı geçişleri meslektaşlarınızın makinelerinde, test makinelerinizde ve nihayetinde üretim makinelerinizde çalıştırmalısınız.

Not:
Uygulama bazında, geçişleri içeren paketin adını ayarı değiştirerek geçersiz kılmak mümkündür.(**migration_modules**)

Geçiş işlemleri aynı veri kümesi üzerinde aynı şekilde çalışacak ve tutarlı sonuçlar üretecektir; yani geliştirme ve test ortamlarında gördüğünüz şey, aynı koşullar altında üretim ortamında da aynen gerçekleşecektir.

Django, modellerinizde veya alanlarınızda yapılan her türlü değişiklik için (veritabanını etkilemeyen seçenekler de dahil) geçişler oluşturur; çünkü bir alanı doğru şekilde yeniden oluşturmasının tek yolu, tüm değişikliklerin geçmişte kayıtlı olmasıdır ve bu seçeneklere daha sonra bazı veri geçişlerinde (örneğin, özel doğrulayıcılar ayarladıysanız) ihtiyacınız olabilir.

Arka Uç Desteği

Django'nun birlikte sunduğu tüm arka uçlarda ve şema değişikliğini (SchemaEditor sınıfı aracılığıyla) destekleyecek şekilde programlanmış üçüncü taraf arka uçlarda da migrasyonlar desteklenir .

Ancak, şema geçişleri söz konusu olduğunda bazı veritabanları diğerlerinden daha yeteneklidir; dikkat edilmesi gereken bazı noktalar aşağıda açıklanmıştır.

PostgreSQL

PostgreSQL, şema desteği açısından buradaki tüm veritabanları arasında en yetenekli olanıdır.

MySQL

MySQL, şema değiştirme işlemleri etrafında işlem desteği sağlamaz; bu da bir geçiş işlemi uygulanamazsa, tekrar denemek için değişiklikleri manuel olarak geri almanız gerektiği anlamına gelir (daha önceki bir noktaya geri dönmek mümkün değildir).

MySQL 8.0, DDL işlemleri için önemli performans iyileştirmeleri getirerek , bu işlemleri daha verimli hale getirdi ve tam tablo yeniden oluşturma ihtiyacını azalttı. Ancak, kilitlenmelerin veya kesintilerin tamamen ortadan kalkmasını garanti edemez. Kilitlenmenin hala gerekli olduğu durumlarda, bu işlemlerin süresi ilgili satır sayısıyla orantılı olacaktır.

Son olarak, MySQL'in bir indeksin kapsadığı tüm sütunların toplam boyutu konusunda nispeten küçük bir sınırı vardır. Bu, diğer arka uçlarda mümkün olan indekslerin MySQL altında oluşturulamayacağı anlamına gelir.

SQLite

SQLite'ın yerleşik şema değiştirme desteği çok sınırlıdır, bu nedenle Django bunu şu şekilde taklit etmeye çalışır:

-Yeni şema ile yeni bir tablo oluşturuluyor.

-Verilerin kopyalanması

-Eski masayı devirmek

-Yeni tablonun adını orijinal adla eşleşecek şekilde değiştirme.

Bu işlem genellikle iyi çalışır, ancak yavaş ve zaman zaman hatalı olabilir. Risklerin ve sınırlamaların çok iyi farkında olmadığınız sürece, SQLite'ı üretim ortamında çalıştırmanız ve taşımanız önerilmez; Django'nun sağladığı destek, geliştiricilerin tam bir veritabanına ihtiyaç duymadan daha az karmaşık Django projeleri geliştirmek için yerel makinelerinde SQLite kullanmalarına olanak sağlamak üzere tasarlanmıştır.

İş Akışı

Django sizin için geçiş dosyaları oluşturabilir. Modellerinizde değişiklikler yapın - örneğin, bir alan ekleyin ve bir modeli kaldırın - ve ardından şunu çalıştırın **makemigrations:**

```
$ python manage.py makemigrations

Migrations for 'books':

 books/migrations/0003_auto.py:

 ~ Alter field author on book
```

Modelleriniz taranacak ve mevcut geçiş dosyalarınızdaki sürümlerle karşılaştırıkacak, ardından yeni bir geçiş sati yazılacaktır. Değişikliklerinizin ne olduğunu anlamak için çıktığı okuduğunuzdan emin olun; mükemmel değil ve karmaşık değişiklikler için beklediğiniz sonucu algılamayabilir.

Yeni geçiş dosyalarınızı aldıktan sonra, beklendiği gibi çalıştıklarından emin olmak için bunları veritabanınıza uygulamanız gerekir:(makemigrations)

```
$ python manage.py migrate

Operations to perform:

 Apply all migrations: books

Running migrations:

 Applying books.0003_auto... OK
```

Geçiş işlemi uygulandıktan sonra, geçişi ve model değişikliklerini sürüm kontrol sisteminize tek bir commit olarak kaydedin; bu sayede diğer geliştiriciler (veya üretim sunucularınız) kodu kontrol ettiklerinde, hem modellerinizdeki değişiklikleri hem de eşlik eden geçişi aynı anda alacaklardır.

Eğer geçiş dosyalarına otomatik olarak oluşturulmuş bir ad yerine anlamlı bir ad vermek istiyorsanız, şu seçeneği kullanabilirsiniz:(makemigrations --name)

```
$ python manage.py makemigrations --name changed_my_model
your_app_label
```

Sürüm kontrolü

Geçişler sürüm kontrolünde saklandığı için, bazen siz ve başka bir geliştiricinin aynı anda aynı uygulamaya bir geçiş işlemi uyguladığı ve bunun sonucunda aynı numaraya sahip iki geçiş işlemi oluştuğu durumlarla karşılaşabilirsiniz.

Merak etmeyin, sayılar sadece geliştiricilerin referansı içindir, Django'nun önemsediği tek şey her geçişin farklı bir isme sahip olmasıdır. Geçişler, aynı uygulamadaki önceki geçişler de dahil olmak üzere, hangi diğer geçişlere bağımlı olduklarını dosyada belirtir; bu nedenle, aynı uygulama için sıralanmamış iki yeni geçiş olduğunda bunu tespit etmek mümkündür.

Bu durumda Django size bir uyarı verecek ve bazı seçenekler sunacaktır. Eğer yeterince güvenli olduğunu düşünürse, iki geçiş dosyasını otomatik olarak doğrusal hale getirmeyi teklif edecektir. Aksi takdirde, geçiş dosyalarını kendiniz değiştirmeniz gerekecektir - endişelenmeyin, bu zor değil ve aşağıdaki Geçiş Dosyaları bölümünde daha ayrıntılı olarak açıklanmıştır .

İşlemler

DDL işlemlerini destekleyen veritabanlarında (SQLite ve PostgreSQL), tüm geçiş işlemleri varsayılan olarak tek bir işlem içinde çalışır. Buna karşılık, bir veritabanı DDL işlemlerini desteklemiyorsa (örneğin MySQL, Oracle), tüm işlemler işlem olmadan çalışır.

atomicBir geçiş işleminin bir işlem içinde çalışmasını, ilgili özniteliği olarak ayarlayarak engelleyebilirsiniz **False**. Örneğin:

```
from Django.db import migrations

class Migration(migrations.Migration):

    atomic = False
```

Ayrıca, `transition` kullanarak veya ` transition` parametresini **atomic()** geçirerek geçişin bazı kısımlarını bir işlem içinde yürütmek de mümkündür . Daha fazla bilgi için Atomik Olmayan Geçişler bölümüne bakın . **atomic=TrueRunPython**

Bağımlılıklar

Veritabanı geçişleri uygulama bazında olsa da, modellerinizin ima ettiği tablolar ve ilişkiler, tek bir uygulama için ayrı ayrı oluşturulamayacak kadar karmaşıktır. Başka bir şeyin çalışmasını gerektiren bir geçiş oluşturduğunuzda (örneğin, **ForeignKey** uygulamanıza bir şey eklediğinizde **books**) **authors**, ortaya çıkan geçiş, içindeki bir geçişe bağımlılık içerecektir **authors**.

Bu, geçişleri çalıştırdığınızda, **authors** önce geçişin çalışıp referans verdiği tabloyu oluşturduğu **ForeignKey**, ardından sütunu oluşturan geçişin **ForeignKey** çalışıp kısıtlamayı oluşturduğu anlamına gelir. Eğer bu gerçekleşmeseydi, geçiş, **ForeignKey** referans verdiği tablo mevcut olmadan sütunu oluşturmaya çalışırdı ve veritabanınız bir hata verirdi.

Bu bağımlılık davranışı, tek bir uygulamayla sınırlama getirdiğiniz çoğu geçiş işlemini etkiler. Tek bir uygulamayla sınırlama (ister içinde ister içinde **makemigrations** olsun **migrate**) en iyi çabayı gösterme sözüdür, bir garanti değildir; bağımlılıkların doğru şekilde elde edilmesi için kullanılması gereken diğer uygulamalar kullanılacaktır.

ForeignKey Geçiş (migration) işlemi yapılmamış uygulamaların, geçiş işlemi yapılmış uygulamalarla ilişkisi (, , vb.) olmamalıdır **ManyToManyField**. Bazen çalışabilir, ancak desteklenmemektedir.

Değiştirilebilir bağımlılıklar

django.db.migrations.swappable_dependency (value) ¶
Bu **swappable_dependency()** fonksiyon, uygulama geçişlerinde, değiştirilen modelin uygulamasındaki geçişlere "değiştirilebilir" bağımlılıklar bildirmek için kullanılır; şu anda bu uygulamanın ilk geçişinde kullanılmaktadır. Sonuç olarak, değiştirilen model ilk geçişte oluşturulmalıdır. Argüman, bir uygulama etiketini ve bir model adını tanımlayan **value** bir dizedir ,
örneğin ."**<app label>.<model>""myapp.MyModel"**

`<password>` kullanarak **swappable_dependency()**, geçiş çerçevesine, geçişin değiştirilebilir bir model oluşturan başka bir geçişe bağlı olduğunu bildirirsiniz; bu da gelecekte modelin farklı bir uygulamayla değiştirilmesi olasılığını sağlar. Bu genellikle, Django'nun kimlik doğrulama sistemindeki özel kullanıcı modeli (**settings.AUTH_USER_MODEL** varsayılan olarak `<password> `) gibi özelleştirmeye veya değiştirmeye tabi olan modelleri referans almak için kullanılır."**auth.User"**

Göç dosyaları

Geçişler, burada "geçiş dosyaları" olarak adlandırılan, disk üzerinde bir formatta saklanır. Bu dosyalar aslında, üzerinde anlaşılmış bir nesne düzenine sahip ve bildirimsel bir tarzda yazılmış normal Python dosyalarıdır.

Temel bir geçiş dosyası şu şekilde görünür:

```
from django.db import migrations, models

class Migration(migrations.Migration):

    dependencies = [("migrations", "0001_initial")]

    operations = [

        migrations.DeleteModel("Tribble"),
        migrations.AddField("Author", "rating",

models.IntegerField(default=0)),

    ]
```

Django, bir geçiş dosyasını (Python modülü olarak) yüklediğinde aradığı şey, django.db.migrations.Migration adlı bir alt sınıftır. Daha sonra bu nesneyi dört öznitelik açısından inceler; bunlardan yalnızca ikisi çoğu zaman kullanılır:

-Dependencis= Bu göçün bağlı olduğu göçlerin bir listesi.

-OperationsOperation= Bu geçiş işleminin ne yaptığını tanımlayan sınıfların bir listesi.

Işlemler kilit noktadır;bunlar, Django'ya hangi şema değişiklerinin yapılması gerektiğini söyleyen bir dizi bildirimsel talimattır. Django bunları tarar ve tüm uygulamalara yönelik tüm şema değişikliklerinin bellekte bir temsilini oluşturur ve bunu şema değişikliklerini yapan SQL' i üretmek için kullanır.

Bellekteki bu yapı, modelleriniz ile geçiş dosyalarınızın mevcut durumu arasındaki farkları belirlemek için de kullanılır; Django, en son çalıştırdığınızda modellerinizin durumunu belirlemek için bellekteki bir model kümesi üzerinde tüm değişiklikleri sırayla işler **makemigrations**. Ardından, dosyalarınızdaki modellerle karşılaştırmak için bu modelleri kullanır ve **models.py** neleri değiştirdiğinizi belirler.

Geçiş dosyalarını elle düzenlemenize nadiren, hatta hiç gerek kalmayacaktır, ancak gerekirse bunları manuel olarak yazmanız tamamen mümkündür. Daha karmaşık işlemlerin bazıları otomatik olarak algılanmaz ve yalnızca elle yazılmış bir geçiş dosyası aracılığıyla kullanılabilir, bu nedenle gerekirse bunları düzenlemekten korkmayın.

Özel alanlar

Zaten geçişi yapılmış özel bir alandaki konumlu bağımsız değişken sayısını, bir hata vermeden değiştiremezsiniz **TypeError**. Eski geçiş, değiştirilmiş yöntemi eski imza ile çağıracaktır **__init__**. Bu nedenle, yeni bir bağımsız değişkene ihtiyacınız varsa, lütfen bir anahtar kelime bağımsız değişkeni oluşturun ve yapıcıya şuna benzer bir şey ekleyin.**assert 'argument_name' in kwargs**

Model yöneticiler

İsteğe bağlı olarak yöneticileri migration'lara serileştirebilir ve işlemlerde kullanılabilir hale getirebilirsiniz . Bu , yönetici sınıfında **RunPython** bir öznitelik tanımlayarak yapılır :**use_in_migrations**

```
class MyManager(models.Manager):

    use_in_migrations = True

class MyModel(models.Model):

    objects = MyManager()
```

Eğer bu fonksiyonu dinamik olarak bir yönetici sınıfı oluşturmak için kullanıyorsanız **from_queryset()**, içe aktarılabilir hale getirmek için oluşturulan sınıftan miras almanız gerekir:

```
class MyManager(MyBaseManager.from_queryset(CustomQuerySet)):

    use_in_migrations = True


class MyModel(models.Model):

    objects = MyManager()
```

Göçlerdeki tarihsel modellerle ilgili notlara bakarak ortaya çıkan sonuçları inceleyebilirsiniz.

İlk göçler

`Göç. başlangıç`

Bir uygulamanın "ilk geçişleri", o uygulamanın tablolarının ilk sürümünü oluşturan geçişlerdir. Genellikle bir uygulamanın bir ilk geçişi olur, ancak karmaşık model bağımlılıkları durumunda iki veya daha fazla geçişi olabilir.

İlk geçişler, geçiş sınıfında bir sınıf özniteliği ile işaretlenir . Bir sınıf özniteliği bulunmazsa, bir geçiş, uygulamadaki ilk geçiş ise (yani aynı uygulamadaki başka bir geçişe bağımlılığı yoksa) "ilk" geçiş olarak kabul edilir.**initial = Trueinitial**

Bu seçenek kullanıldığında, bu ilk geçişler özel olarak ele alınır. Bir veya daha fazla tablo oluşturan (işlem) bir ilk geçiş için Django, bu tabloların tümünün veritabanında zaten mevcut olup olmadığını kontrol eder ve mevcutsa geçişi sahte olarak uygular. Benzer şekilde, bir veya daha fazla alan ekleyen (işlem) bir ilk geçiş için Django, ilgili sütunların tümünün veritabanında zaten mevcut olup olmadığını kontrol eder ve mevcutsa geçişi sahte olarak uygular. Bu seçenek kullanılmadığında , ilk geçişler diğer geçişlerden farklı şekilde ele alınmaz.**migrate --fake-initialCreateModelAddField--fake-initial**

Tarihsel tutarlılık

Daha önce de belirtildiği gibi, iki geliştirme dalı birleştirildiğinde geçişleri manuel olarak doğrusallaştırmanız gerekebilir. Geçiş bağımlılıklarını düzenlerken, bir geçişin uygulandığı ancak bazı bağımlılıklarının uygulanmadığı tutarsız bir geçmiş durumu oluşturabilirsiniz. Bu, bağımlılıkların yanlış olduğuna dair güçlü bir göstergedir, bu nedenle Django, düzeltilene kadar geçişleri çalıştırmayı veya yeni geçişler oluşturmayı reddedecektir. Birden fazla veritabanı kullanırken, hangi veritabanlarının tutarlı geçmişi kontrol edeceğini kontrol etmek için veritabanı yönlendiricileri**allow_migrate()** yöntemini kullanabilirsiniz .**makemigrations**

Uygulamalara geçiş dosyaları ekleme

makemigrationsYeni uygulamalar, geçişleri kabul edecek şekilde önceden yapılandırılmış olarak gelir; bu nedenle , bazı değişiklikler yaptıktan sonra bir kez çalıştırarak geçişler ekleyebilirsiniz .

Eğer uygulamanızda zaten modeller ve veritabanı tabloları varsa ve henüz migration'lar yoksa (örneğin, önceki bir Django sürümüne göre oluşturduysanız), migration'ları kullanacak şekilde dönüştürmek için şu komutu çalıştırmanız gerekecektir:

```
$ python manage.py makemigrations your_app_label
```

Bu, uygulamanız için yeni bir başlangıç geçişi oluşturacaktır. Şimdi, komutunu çalıştırın ve Django, bir başlangıç geçişiniz olduğunu *ve* oluşturmak istediği tabloların zaten mevcut olduğunu algılayacak ve geçişi zaten uygulanmış olarak işaretleyecektir. (Bu işaret olmadan, komut hata verecektir çünkü oluşturmak istediği tablolar zaten mevcuttur.) **python manage.py migrate --fake-initialmigrate --fake-initial**. Unutmayın ki bu yöntem yalnızca iki koşul sağlandığında işe yarar:

-Tabloları oluşturduğunuzdan beri modellerinizde herhangi bir değişiklik yapmadınız. Geçişlerin çalışması için *önce* ilk geçişi yapmalı , ardından değişiklikleri yapmalısınız, çünkü Django değişiklikleri veritabanıyla değil, geçiş dosyalarıyla karşılaştırır.

-Veritabanınızı manuel olarak düzenlemediniz - Django, veritabanınızın modellerinizle eşleşmediğini algılayamaz, yalnızca geçişler bu tabloları değiştirmeye çalıştığında hatalar alırsınız.

Göçlerin tersine çevrilmesi

migrateÖnceki geçişin numarasını girerek geçişler geri alınabilir . Örneğin, geçişi geri almak için **books.0003**

```
$ python manage.py migrate books 0002
Operations to perform:
  Target specific migration: 0002_auto, from books
Running migrations:
  Rendering model states... DONE
  Unapplying books.0003_auto... OK
```

Bir uygulama için uygulanan tüm geçişleri geri almak istiyorsanız, şu adı kullanın **zero**:

```
$ python manage.py migrate books zero
Operations to perform:
  Unapply all migrations: books
Running migrations:
  Rendering model states... DONE
  Unapplying books.0002_auto... OK
  Unapplying books.0001_initial... OK
```

Bir geçiş işlemi, geri döndürülemez işlemler içeriyorsa geri döndürülemezdir. Bu tür geçiş işlemlerini geri almaya çalışmak şu hatayı verecektir **IrreversibleError**:

```
$ python manage.py migrate books 0002
Operations to perform:
  Target specific migration: 0002_auto, from books
Running migrations:
   Rendering model states... DONE
  Unapplying books.0003_auto...Traceback (most recent call last):
django.db.migrations.exceptions.IrreversibleError:
 Operation <RunSQL  sql='DROP TABLE demo_books'> in
 books.0003_auto is not reversible
```

Tarihsel modeller

Veritabanı geçişlerini çalıştırdığınızda, Django geçiş dosyalarında saklanan modellerinizin geçmiş sürümlerinden çalışır. Eğer bu işlemi kullanarak Python kodu yazıyorsanız veya veritabanı yönlendiricilerinizde yöntemleriniz **RunPython** varsa , modelleri doğrudan içe aktarmak yerine bu geçmiş model sürümlerini **kullanmanız gerekir .allow_migrate**

Uyarı:
Geçmişe ait modelleri kullanmak yerine modelleri doğrudan içe aktarırsanız, geçişleriniz başlangıçta çalışabilir ancak gelecekte eski geçişleri yeniden çalıştırmaya çalıştığınızda (genellikle yeni bir kurulum yaptığınızda ve veritabanını kurmak için tüm geçişleri çalıştırdığınızda) başarısız olacaktır.
Bu, geçmişe ait model sorunlarının hemen fark edilmeyebileceği anlamına gelir. Bu tür bir hatayla karşılaşırsanız, doğrudan içe aktarmalar yerine geçmişe ait modelleri kullanacak şekilde geçiş dosyasını düzenleyip bu değişiklikleri kaydetmeniz sorun teşkil etmez.

Rastgele Python kodunu serileştirmek imkansız olduğundan, bu geçmişe ait modellerde sizin tanımladığınız özel yöntemler bulunmayacaktır. Bununla birlikte, aynı alanlara, ilişkilere, yöneticilere (yalnızca belirli özelliklere sahip olanlarla sınırlı) ve seçeneklere (ayrıca sürümlü olduklarından mevcut seçeneklerinizden farklı olabilirler) sahip olacaklardır.**use_in_migrations = TrueMeta**

Uyarı:
Bu, geçişlerde nesnelere erişirken özel yöntemlerin çağrılmayacağı **save()** ve özel kurucu veya örnek yöntemlerin bulunmayacağı anlamına gelir. Buna göre plan yapın!

Alan seçeneklerindeki işlevlere yapılan referanslar **upload_to**ve **limit_choices_to** yöneticilerin bulunduğu model yöneticisi bildirimleri geçişlerde serileştirilir, bu nedenle işlevler ve sınıflar, onlara referans veren bir geçiş olduğu sürece saklanmalıdır. Özel model alanları da saklanmalıdır, çünkü bunlar geçişler tarafından doğrudan içe aktarılır.**use_in_migrations = True**
Ek olarak, modelin somut temel sınıfları işaretçiler olarak saklanır, bu nedenle onlara referans içeren bir geçiş (migration) olduğu sürece temel sınıfları her zaman saklamanız gerekir. Olumlu tarafı ise, bu temel sınıflardaki yöntemler ve yöneticiler normal şekilde miras alır, bu nedenle bunlara kesinlikle erişmeniz gerekiyorsa, bunları bir üst sınıfa taşımayı tercih edebilirsiniz.
Eski referansları kaldırmak için, migration dosyalarını sıkıştırabilir veya çok fazla referans yoksa bunları migration dosyalarına kopyalayabilirsiniz.

Model alanlarını kaldırırken dikkate alınması gerekenler

Önceki bölümde açıklanan "geçmişteki işlevlere yapılan atıflar" hususlarına benzer şekilde, özel model alanlarını projenizden veya üçüncü taraf uygulamanızdan kaldırmak, eski geçişlerde bunlara referans verilmesi durumunda sorun yaratacaktır.

Bu durumu kolaylaştırmak için Django, sistem kontrolleri çerçevesini kullanarak model alanlarının kullanım dışı bırakılmasına yardımcı olacak bazı model alanı öznitelikleri sunar .

system_check_deprecated_details Aşağıdaki gibi, özniteliği model alanınıza ekleyin :

```
class IPAddressField(Field):

    system_check_deprecated_details = {

        "msg": (

            "IPAddressField has been deprecated. Support for it
(except "

            "in historical migrations) will be removed in Django
1.9."

        ),

        "hint": "Use GenericIPAddressField instead.",  # optional

        "id": "fields.W900",  # pick a unique ID for your field.

    }
```

İstediğiniz bir süre (Django'nun kendi alanları için iki veya üç özellik sürümü) geçtikten sonra, **system_check_deprecated_details** özniteliği değiştirin **system_check_removed_details** ve sözlüğü aşağıdaki gibi güncelleyin:

```
class IPAddressField(Field):

    system_check_removed_details = {

        "msg": (

            "IPAddressField has been removed except for support in
"

            "historical migrations."

        ),

        "hint": "Use GenericIPAddressField instead.",

        "id": "fields.E900",  # pick a unique ID for your field.

    }
```

__init__() Veritabanı geçişlerinde çalışması için gerekli olan alanın yöntemlerini (örneğin , **deconstruct()**, ve) korumalısınız **get_internal_type()**. Bu alanı, alanı referans alan tüm geçişler var olduğu sürece saklayın. Örneğin, geçişleri birleştirdikten ve eskilerini kaldırdıktan sonra, alanı tamamen kaldırabilirsiniz.

Veri Geçişleri

Veritabanı şemasını değiştirmenin yanı sıra, isterseniz şemayla birlikte veritabanındaki verileri değiştirmek için de geçiş işlemlerini kullanabilirsiniz.

Verileri değiştiren geçiş işlemleri genellikle "veri geçişleri" olarak adlandırılır; bunlar şema geçişlerinizin yanında ayrı geçişler olarak yazılmalıdır.

Django, şema geçişlerinde olduğu gibi veri geçişlerini otomatik olarak oluşturamaz, ancak bunları yazmak çok zor değildir. Django'daki geçiş dosyaları İşlemlerden oluşur ve veri geçişleri için kullandığınız ana işlem şudur **RunPython**: .

Öncelikle, üzerinde çalışabileceğiniz boş bir migration dosyası oluşturun (Django dosyayı doğru yere yerleştirecek, bir isim önerecek ve bağımlılıkları sizin için ekleyecektir):

python manage.py makemigrations --empty yourappname

Ardından dosyayı açın; dosya aşağıdakine benzer bir şey olmalı:

```
# Generated by Django A.B on YYYY-MM-DD HH:MMfrom django.db import
migrations

class Migration(migrations.Migration):

    dependencies = [

        ("yourappname", "0001_initial"),

    ]

     operations = []
```

Şimdi yapmanız gereken tek şey yeni bir fonksiyon oluşturmak ve **RunPython** onu kullanmak.
Bu **RunPython** fonksiyon, iki argüman alan çağrılabilir bir nesne bekler; birincisi, geçmişinizdeki geçişin bulunduğu yere uyması için tüm modellerinizin geçmiş sürümlerini içeren bir uygulama kayıt defteridir ve ikincisi , veritabanı şema değişikliklerini manuel olarak gerçekleştirmek için kullanabileceğiniz bir SchemaEditor'dur (ancak dikkat edin, bunu yapmak geçiş otomatik algılayıcısını karıştırabilir).

Şimdi, yeni alanımızı ve **name** değerlerinin birleştirilmiş haliyle dolduracak bir migration yazalım (artık aklımız başımızda ve herkesin ad ve soyadı olmadığını fark ettik). Tek yapmamız gereken, historical modelini kullanmak ve satırlar üzerinde yineleme yapmak:**first_namelast_name**

```
from django.db import migrations

def combine_names(apps, schema_editor):

    # We can't import the Person model directly as it may be a
newer

    # version than this migration expects. We use the historical
version.

    Person = apps.get_model("yourappname", "Person")

    for person in Person.objects.all():

        person.name = f"{person.first_name} {person.last_name}"

        person.save()

class Migration(migrations.Migration):

    dependencies = [

        ("yourappname", "0001_initial"),

    ]

     operations = [

        migrations.RunPython(combine_names),

    ]
```

Bu işlem tamamlandıktan sonra, normal şekilde çalışmaya devam edebiliriz ve veri geçişi diğer geçişlerle birlikte yerinde gerçekleşecektir.**python manage.py migrate**

Geriye doğru geçiş yaparken çalıştırmak istediğiniz mantığı yürütmek için ikinci bir çağrılabilir fonksiyon geçirebilirsiniz **RunPython**. Bu çağrılabilir fonksiyon atlanırsa, geriye doğru geçiş bir istisna hatası verecektir.

Diğer uygulamalardan modellere erişme ¶

Bir fonksiyon yazarken ve **RunPython** bu fonksiyon, migration dosyasının bulunduğu uygulama dışındaki uygulamalardan modeller kullanıyorsa, migration özniteliği ilgili her uygulamanın en son migration dosyasını içermelidir; aksi takdirde, fonksiyon içinde modeli . kullanarak almaya çalıştığınızda **dependencies** şuna benzer bir hata alabilirsiniz:**LookupError: No installed app with label 'myappname'RunPythonapps.get_model()**

Aşağıdaki örnekte, **app1** içinde bulunan modelleri kullanması gereken bir geçiş işlemimiz var **app2**. 'nin ayrıntılarıyla ilgilenmiyoruz, **move_m1** sadece her iki uygulamadaki modellere erişmesi gerekeceği gerçeğiyle ilgileniyoruz. Bu nedenle, 'nin son geçiş işlemini belirten bir bağımlılık ekledik **app2**:

```
class Migration(migrations.Migration):

    dependencies = [

        ("app1", "0001_initial"),

        # added dependency to enable using models from app2 in
move_m1

        ("app2", "0004_foobar"),

    ]

    operations = [

        migrations.RunPython(move_m1),

    ]
```

Daha gelişmiş geçişler

Daha gelişmiş geçiş işlemleriyle ilgileniyorsanız veya kendi geçiş işlemlerinizi yazmak istiyorsanız, geçiş işlemleri referansına ve geçiş yazma kılavuzuna bakın .

Göçleri bastırmak

Veritabanı geçişlerini (migration) özgürce yapmanız ve kaç tane olduğu konusunda endişelenmemeniz teşvik edilir; geçiş kodu, çok fazla yavaşlama olmadan yüzlerce geçişi aynı anda işleyebilecek şekilde optimize edilmiştir. Ancak, sonunda birkaç yüz geçişten sadece birkaçına geri dönmek isteyeceksiniz ve işte burada sıkıştırma (squashing) devreye giriyor.

Sıkıştırarak birleştirme, mevcut birçok göç işlemini aynı değişiklikleri temsil eden tek bir (veya bazen birkaç) göç işlemine indirgeme işlemidir.

Django bunu, mevcut tüm geçiş dosyalarınızı alarak, içlerindeki **Operation** öğeleri çıkarıp sıraya koyarak ve ardından listenin uzunluğunu azaltmaya çalışmak için bunlar üzerinde bir optimizasyon algoritması çalıştırarak yapar; örneğin, ve'nin birbirini götürdüğünü ve'nin **CreateModel** içine dahil edilebileceğini bilir .**DeleteModelAddFieldCreateModel**

İşlem dizisi olabildiğince kısaltıldıktan sonra (mümkün olan miktar, modellerinizin ne kadar iç içe geçmiş olduğuna ve herhangi bir işlem olup olmadığına bağlıdır **RunSQL** (**RunPython** bunlar, olarak işaretlenmedikçe optimize edilemez **elidable**)), Django bunu yeni bir dizi geçiş dosyasına geri yazacaktır.

Bu dosyalar, daha önce sıkıştırılmış olan geçiş dosyalarının yerini alacak şekilde işaretlenmiştir, böylece eski geçiş dosyalarıyla birlikte var olabilirler ve Django, geçmişte nerede olduğunuza bağlı olarak aralarında akıllıca geçiş yapacaktır. Sıkıştırma işleminde henüz ilerlemediyseniz, sona ulaşana kadar bunları kullanmaya devam edecek ve ardından sıkıştırılmış geçmişe geçecektir; yeni kurulumlar ise yeni sıkıştırılmış geçiş dosyasını kullanacak ve eski olanların tümünü atlayacaktır.

Bu sayede, henüz tam olarak güncellenmemiş, üretimde olan sistemleri bozmadan eski dosyaları sıkıştırabilirsiniz. Önerilen süreç, eski dosyaları koruyarak sıkıştırmak, değişiklikleri kaydetmek ve yayınlamak, tüm sistemler yeni sürümle güncellenene kadar beklemek (veya üçüncü taraf bir proje iseniz, kullanıcılarınızın herhangi bir sürümü atlamadan sürümleri sırayla yükseltmelerini sağlamak) ve ardından eski dosyaları kaldırmak, değişiklikleri kaydetmek ve ikinci bir sürüm yayınlamaktır.

Tüm bu işlemleri destekleyen komut şu şekildedir: **squashmigrations** Birleştirmek istediğiniz uygulama etiketini ve geçiş adını ona iletin, gerisini o halleder:

```
$ ./manage.py squashmigrations myapp 0004


Will squash the following migrations: - 0001_initial -
0002_some_change - 0003_another_change - 0004_undo_something


Do you wish to proceed? [y/N] y


Optimizing...  Optimized from 12 operations to 7 operations.


Created new squashed migration
/home/andrew/Programs/DjangoTest/test/migrations/0001_squashed_000
4_undo_something.py  You should commit this migration but leave
the old ones in place;  the new migration will be used for new
installs. Once you are sure  all instances of the codebase have
applied the migrations you squashed,  you can delete them.
```

Sıkıştırılmış geçiş dosyasının adını otomatik olarak oluşturulan bir ad yerine kendiniz belirlemek istiyorsanız bu seçeneği kullanın .**squashmigrations --squashed-name**

Django'da model bağımlılıklarının çok karmaşık olabileceğini ve birleştirmenin (squashing) çalışmama ile sonuçlanabileceğini unutmayın; bu durum ya yanlış optimize edilmiş (bu durumda tekrar deneyebilirsiniz **--no-optimize**, ancak bir sorun da bildirmelisiniz) ya da bir hata (.g.h.) ile **CircularDependencyError** sonuçlanabilir; bu durumda sorunu manuel olarak çözebilirsiniz.

Bir sorunu manuel olarak çözmek için **CircularDependencyError**, döngüsel bağımlılıktaki ForeignKey'lerden birini ayrı bir migration'a ayırın ve diğer uygulamaya olan bağımlılığı da onunla birlikte taşıyın. Emin değilseniz, **makemigrations** modellerinizden tamamen yeni migration'lar oluşturmanız istendiğinde bu sorunu nasıl ele aldığını inceleyin. Django'nun gelecekteki bir sürümünde, **squashmigrations** bu hataları kendi başına çözmeye çalışacak şekilde güncellenecektir.

Geçiş işlemini tamamladıktan sonra, değiştirdiği geçiş işlemleriyle birlikte bu değişikliği kaydetmeli ve uygulamanızın çalışan tüm örneklerine dağıtmalısınız. **Migrate** Değişikliğin veritabanlarında saklanması için bu örneklerin çalıştırıldığından emin olmalısınız.

Ardından, sıkıştırılmış geçiş işlemini aşağıdaki adımları izleyerek normal bir geçiş işlemine dönüştürebilirsiniz:

-Yerine geçtiği tüm geçiş dosyalarını siliyor.

-Silinen geçiş dosyalarına bağlı olan tüm geçiş dosyaları, bunun yerine birleştirilen geçiş dosyasına bağlı olacak şekilde güncelleniyor.

-Sıkıştırılmış geçiş dosyasının sınıfındaki **replaces** özniteliği kaldırmak (Django, bunun sıkıştırılmış bir geçiş dosyası olduğunu bu şekilde anlar).**Migration**

Not

Sıkıştırılmış geçiş dosyalarını normal geçiş dosyalarına dönüştürmeden de sıkıştırabilirsiniz; bu, her ortamın henüz orijinal sıkıştırılmış geçiş kümesini çalıştırmadığı durumlarda yararlı olabilir. Ancak genel olarak, eski geçiş dosyalarını temizleyebilmek için sıkıştırılmış geçiş dosyalarını normal geçiş dosyalarına dönüştürmek daha iyidir.

Silinen geçişlere yapılan referansların budanması

Silinen bir migration dosyasının adını gelecekte tekrar kullanma olasılığınız varsa, bu dosyayı Django'nun migrations tablosundan ilgili seçenekle kaldırmanız gerekir.

Değerlerin serileştirilmesi

Migrasyonlar, modellerinizin eski tanımlarını içeren Python dosyalarıdır; bu nedenle, migrasyon yazmak için Django'nun modellerinizin mevcut durumunu alıp bir dosyaya serileştirmesi gerekir.

Django çoğu şeyi serileştirebilirken, bazı şeyleri geçerli bir Python gösterimine dönüştüremeyiz; bir değerin nasıl koda geri dönüştürüleceğine dair bir Python standardı yoktur (**repr()** yalnızca temel değerler için geçerlidir ve içe aktarma yollarını belirtmez).

Django aşağıdaki verileri serileştirebilir:

-int, **float**, **bool**, **str**, **bytes**, **None**,**NoneType**

-list, **set**, **tuple**, **dict**,**range**

-datetime.date, **datetime.time**, ve **datetime.datetime** örnekleri (saat dilimine duyarlı olanlar dahil)

-**zoneinfo.ZoneInfo** örnekler

-decimal.Decimal örnekler

-**enum.Enum** ve **enum.Flag** örnekler

-uuid.UUID örnekler

-**functools.partial()** ve serileştirilebilir , ve değerlerine **functools.partialmethod**sahip örnekler**funcargskeywords**

-.NET'ten elde edilen saf ve somut yol nesneleri **pathlib**. Somut yollar, saf yol eşdeğerlerine dönüştürülür, örneğin **pathlib.PosixPath**.NET'e. **pathlib.PurePosixPath**

-**os.PathLike** ,örnekler, örneğin **os.DirEntry**, dönüştürülen **str** veya **bytes** kullanılanlar**os.fspath()**

-LazyObject ,Serileştirilebilir bir değeri saran örnekler

-Numaralandırma türleri (örneğin **TextChoices**veya **IntegerChoices**) örnekleri

-Herhangi bir Django alanı

-Herhangi bir fonksiyon veya metot referansı (örneğin **datetime.datetime.today**) (modülün en üst düzey kapsamı içinde olmalıdır)

---İşlevler, uygun şekilde paketlenirse, yani şu yöntem kullanılırsa, süslenebilir: **functools.wraps()**

---Dekoratörler açıkça desteklenmektedir **functools.cache()**.**functools.lru_cache()**

-Sınıf gövdesi içinden kullanılan bağlı olmayan yöntemler

-Herhangi bir sınıf referansı (modülün en üst düzey kapsamı içinde olmalıdır)

-Özel bir **deconstruct()** yöntemi olan her şey (aşağıya bakınız)

Django seri hale getiremiyor:

-İç içe sınıflar

-Rastgele sınıf örnekleri (ör.)**MyClass(4.3, 5.7)**

-Lambdalar

Özel seri numaralandırıcılar

Özel bir seri hale getirici yazarak diğer türleri de seri hale getirebilirsiniz. Örneğin, Django varsayılan olarak seri hale getirme yapmıyorsa **Decimal**, şunu yapabilirsiniz:

```
from decimal import Decimal

from django.db.migrations.serializer import BaseSerializerfrom
django.db.migrations.writer import MigrationWriter

class DecimalSerializer(BaseSerializer):

    def serialize(self):

        return repr(self.value), {"from decimal import Decimal"}

MigrationWriter.register_serializer(Decimal, DecimalSerializer)
```

İlk argüman, **MigrationWriter.register_serializer()** serileştiriciyi kullanması gereken bir tür veya türlerin yinelenebilir bir listesidir.

Serileştiricinizin yöntemi **serialize()**, değerin geçişlerde nasıl görünmesi gerektiğine dair bir dize ve geçişte ihtiyaç duyulan tüm içe aktarmaların bir listesini döndürmelidir.

deconstruct()Bir yöntem ekleme

Django'nun kendi özel sınıf örneklerinizi serileştirmesini sağlamak için sınıfa bir metot verebilirsiniz **deconstruct()**. Bu metot hiçbir argüman almaz ve üç öğeden oluşan bir demet döndürmelidir :**(path, args, kwargs)**

-path=`<class>` ifadesi, sınıfın Python yolunu belirtmeli ve sınıf adı en sonda yer almalıdır (örneğin, `<class> **myapp.custom_things.MyClass**`). Eğer sınıfınız modülün en üst düzeyinde mevcut değilse, serileştirilemez.

-args=Bu, sınıfınızın **__init__**metoduna iletilecek konumlu argümanların bir listesi olmalıdır. Bu listedeki her şeyin kendisi de serileştirilebilir olmalıdır.

-kwargs=Bu, sınıfınızın **__init__**metoduna iletilecek anahtar kelime argümanlarının bir sözlüğü olmalıdır. Her değerin kendisi serileştirilebilir olmalıdır.

Not

Bu dönüş değeri , dört öğeden oluşan bir demet döndüren özel alanlar için **deconstruct()** kullanılan yöntemden farklıdır .

Django, tıpkı Django alanlarına referansları yazdığı gibi, verilen argümanlarla sınıfınızın bir örneğini oluşturarak değeri yazacaktır.

Her çalıştırmada yeni bir geçiş dosyasının oluşturulmasını önlemek için , süslenmiş sınıfa **makemigrations** bir yöntem de eklemelisiniz . Bu fonksiyon, durumlar arasındaki değişiklikleri tespit etmek için Django'nun geçiş çerçevesi tarafından çağrılacaktır.**__eq__()**

Sınıfınızın kurucusuna gönderilen tüm argümanlar kendileri serileştirilebilir olduğu sürece, yöntemi eklemek için **@deconstructible** sınıf dekoratörünü kullanabilirsiniz :**django.utils.deconstructdeconstruct()**

```
from django.utils.deconstruct import deconstructible

@deconstructibleclass MyCustomClass:

    def __init__(self, foo=1):

        self.foo = foo

        ...

    def __eq__(self, other):

        return self.foo == other.foo
```

Bu dekoratör, yapıcıya giden argümanları yakalamak ve korumak için mantık ekler ve ardından deconstruct() çağrıldığında bu argümanları tam olarak döndürür.

Birden fazla Django sürümünü destekleme

Modeller içeren üçüncü taraf bir uygulamanın geliştiricisiyseniz, birden fazla Django sürümünü destekleyen geçiş dosyaları (migrations) göndermeniz gerekebilir. Bu durumda, her zaman **desteklemek istediğiniz en düşük Django sürümüyle** çalışmalısınız .**makemigrations**

Geçiş sistemi, Django'nun geri kalanıyla aynı politikaya göre geriye dönük uyumluluğu koruyacaktır; bu nedenle Django XY'de oluşturulan geçiş dosyaları, Django X.Y+1'de değişmeden çalışmalıdır. Ancak geçiş sistemi ileriye dönük uyumluluk garantisi vermez. Yeni özellikler eklenebilir ve Django'nun daha yeni sürümleriyle oluşturulan geçiş dosyaları eski sürümlerde çalışmayabilir.

Ayrıca bakınız:

--Göç İşlemleri Referansı

Şema işlemleri API'sini, özel işlemleri ve kendi işlemlerinizi yazmayı kapsar.

--Yazma Göçleri "nasıl yapılır" kılavuzu

Karşılaşabileceğiniz farklı senaryolar için veritabanı geçişlerinin nasıl yapılandırılacağını ve yazılacağını açıklar.

Muhammet Emir Ahmet Tofan

Migration Operations (Göç işlemleri)¶
Göç dosyaları, göçün veritabanınıza ne yapması gerektiğini bildirimsel olarak kaydeden bir veya daha fazla İşlem nesnesinden oluşur.

Django ayrıca bu İşlem nesnelerini, modellerinizin geçmişte nasıl göründüğünü belirlemek ve son göçten bu yana modellerinizde yaptığınız değişiklikleri hesaplamak için kullanır, böylece göçlerinizi otomatik olarak yazabilir; bu yüzden bunlar bildirimsel niteliktedir, çünkü Django bunları kolayca belleğe yükleyebilir ve veritabanına dokunmadan çalıştırarak projenizin nasıl görünmesi gerektiğini belirleyebilir.

Veri göçleri ve gelişmiş manuel veritabanı manipülasyonu gibi şeyler için daha özel İşlem nesneleri de vardır. Sık yaptığınız özel bir değişikliği kapsüllemek istiyorsanız, kendi İşlem sınıflarınızı da yazabilirsiniz.

Kendi Operation nesnelerinizi yazmak için boş bir geçiş dosyasına ihtiyacınız varsa, python manage.py makemigrations --empty yourappname komutunu kullanın, ancak şema değiştiren işlemleri manuel olarak eklemenin geçiş otomatik algılayıcısını karıştırabileceğini ve makemigrations komutunun çalıştırılması sonucunda hatalı kod çıktısı verebileceğini unutmayın.

Tüm temel Django işlemleri django.db.migrations.operations modülünden kullanılabilir.

Giriş niteliğinde materyal için, göçler konu kılavuzuna bakın.

Schema Operations (Şema İşlemleri)
class CreateModel(name, fields, options=None, bases=None, managers=None)[source]¶
Proje geçmişinde yeni bir model ve buna karşılık gelen veritabanında bir tablo oluşturur.

name, models.py dosyasında yazıldığı gibi model adıdır.

fields, (field_name, field_instance) çiftlerinden oluşan bir listedir. Alan örneği, bağlı olmayan bir alan olmalıdır (yani başka bir modelden alınan bir alan değil, sadece models.CharField(...) olmalıdır).

options, modelin Meta sınıfından alınan değerlerin isteğe bağlı bir sözlüğüdür.

bases, bu modelin miras alacağı diğer sınıfların isteğe bağlı bir listesidir; başka bir modele bağımlı olmak istiyorsanız (böylece tarihsel sürümden miras alırsınız), hem sınıf nesnelerini hem de "appname.ModelName" biçimindeki dizeleri içerebilir. Sağlanmazsa, varsayılan olarak standart models.Model'den miras alır.

managers, (manager_name, manager_instance) çiftlerinden oluşan bir liste alır. Listedeki ilk yönetici, geçişler sırasında bu modelin varsayılan yöneticisi olacaktır.

DeleteModel¶ (Modeli Sil)
class DeleteModel(name)[source]¶
Modeli proje geçmişinden ve tablosunu veritabanından siler.
RenameModel¶ (Modeli Yeniden Adlandır)
class RenameModel(old_name, new_name)[source]¶
Modelin adını eski adından yeni adına değiştirir.

Modelin adını ve birçok alanını aynı anda değiştirirseniz, bunu manuel olarak eklemeniz gerekebilir; otomatik algılayıcıya göre, bu, eski adıyla bir modeli silip farklı bir adla yeni bir model eklemişsiniz gibi görünecek ve oluşturduğu geçiş, eski tablodaki tüm verileri kaybedecektir.

AlterModelTable¶ (Model Tablosunu Değiştir)
class AlterModelTable(name, table)[source]¶
Modelin tablo adını değiştirir (Meta alt sınıfındaki db_table seçeneği).

AlterModelTableComment¶ (Model Tablosu Yorumunu Değiştir)
class AlterModelTableComment(name, table_comment)[source]¶
Modelin tablo yorumunu değiştirir (Meta alt sınıfındaki db_table_comment seçeneği).

AlterUniqueTogether¶ (Benzersiz Olanı Birlikte Değiştir)
class AlterUniqueTogether(name, unique_together)[source]¶
Modelin benzersiz kısıtlamalar kümesini değiştirir (Meta alt sınıfındaki unique_together seçeneği).

AlterIndexTogether¶ (Endeksi Birlikte Değiştir)
class AlterIndexTogether(name, index_together)[source]¶
Modelin özel dizinler kümesini değiştirir (Meta alt sınıfındaki index_together seçeneği).

Uyarı

AlterIndexTogether resmi olarak yalnızca Django 4.2 öncesi geçiş dosyaları için desteklenir. Geriye dönük uyumluluk nedenleriyle, hala genel API'nın bir parçasıdır ve kullanımdan kaldırılması veya kaldırılması planlanmamaktadır, ancak yeni geçişler için kullanılmamalıdır. Bunun yerine AddIndex ve RemoveIndex işlemlerini kullanın.

AlterOrderWithRespectTo¶ (Sırayı Değiştir)
class AlterOrderWithRespectTo(name, order_with_respect_to)[source]¶
Meta alt sınıfındaki order_with_respect_to seçeneği için gerekli olan _order sütununu oluşturur veya siler.

AlterModelOptions¶ (Model Seçeneklerini Değiştir)
class AlterModelOptions(name, options)[source]
İzinler ve verbose_name gibi çeşitli model seçeneklerindeki (bir modelin Meta'sındaki ayarlar) değişiklikleri depolar. Veritabanını etkilemez, ancak RunPython örneklerinin kullanması için bu değişiklikleri kalıcı hale getirir. options, seçenek adlarını değerlere eşleyen bir sözlük olmalıdır.

AlterModelManagers¶ Model Yöneticilerini Değiştir)
class AlterModelManagers(name, managers)[source]¶
Göçler sırasında kullanılabilen yöneticileri değiştirir.

AddField¶ (Alan Ekle)
class AddField(model_name, name, field, preserve_default=True)[source]¶
Modele bir alan ekler. model_name modelin adı, name alanın adı ve field bağlanmamış bir Field örneğidir (models.py'deki alan bildirimine koyacağınız şey - örneğin, models.IntegerField(null=True).

preserve_default argümanı, alanın varsayılan değerinin kalıcı olup proje durumuna dahil edilmesi gerektiğini (True) veya geçici olup sadece bu geçiş için geçerli olduğunu (False) belirtir.

Genellikle, geçiş işlemi bir tabloya nullable olmayan bir alan eklediği ve mevcut satırlara yerleştirilecek bir varsayılan değer gerektirdiği için olur. Bu, veritabanında varsayılanları ayarlama davranışını doğrudan etkilemez - Django asla veritabanı varsayılanlarını ayarlamaz ve bunları her zaman Django ORM kodunda uygular.

Uyarı

Eski veritabanlarında, varsayılan değeri olan bir alan eklemek tablonun tamamen yeniden yazılmasına neden olabilir. Bu, nullable alanlar için bile geçerlidir ve performans üzerinde olumsuz bir etkiye sahip olabilir. Bunu önlemek için aşağıdaki adımlar atılmalıdır.

Varsayılan değer olmadan nullable alanı ekleyin ve makemigrations komutunu çalıştırın. Bu, AddField işlemiyle bir geçiş oluşturmalıdır.

Alanınıza varsayılan değeri ekleyin ve makemigrations komutunu çalıştırın. Bu, AlterField işlemiyle bir geçiş oluşturmalıdır.

RemoveField¶ (Alanı Kaldır)
class RemoveField(model_name, name)[source]
Bir modelden bir alanı kaldırır.

Tersine çevrildiğinde, bu aslında modele bir alan eklemek anlamına gelir. Alan nullable ise veya yeniden oluşturulan sütunu doldurmak için kullanılabilecek bir

varsayılan değere sahipse, işlem geri alınabilir (geri alınamayan veri kaybı hariç). Alan nullable değilse ve varsayılan bir değere sahip değilse, işlem geri alınamaz.

Django 6.0'da değiştirildi:
BaseDatabaseSchemaEditor ve PostgreSQL arka uçları, görünümler gibi bağımlı ilişkili veritabanı nesnelerini silmek için artık CASCADE kullanmaz. Django tarafından yönetilmeyen bağımlı nesnelerin, RemoveField çalıştırılmadan önce manuel olarak kaldırılması gerekebilir.

AlterField¶ (Alanı Değiştir)
class AlterField(model_name, name, field, preserve_default=True)[source]¶
Bir alanın tanımını değiştirir; buna tür, null, unique, db_column ve diğer alan özniteliklerinde yapılan değişiklikler de dahildir.

preserve_default argümanı, alanın varsayılan değerinin kalıcı olup proje durumuna dahil edilmesi gerektiğini (True) veya geçici olup sadece bu geçiş için geçerli olduğunu (False) belirtir - genellikle geçiş, nullable bir alanı nullable olmayan bir alana dönüştürdüğü ve mevcut satırlara eklenecek bir varsayılan değere ihtiyaç duyduğu için. Veritabanında varsayılanları doğrudan ayarlama davranışını etkilemez - Django asla veritabanı varsayılanlarını ayarlamaz ve bunları her zaman Django ORM kodunda uygular.

Tüm değişikliklerin tüm veritabanlarında mümkün olmadığını unutmayın - örneğin, çoğu veritabanında models.TextField() gibi bir metin türü alanı models.IntegerField() gibi bir sayı türü alanına değiştiremezsiniz.

RenameField¶ (Alanı Yeniden Adlandır)
class RenameField(model_name, old_name, new_name)[source]¶
Bir alanın adını (ve db_column ayarlanmamışsa, sütun adını) değiştirir.

AddIndex¶ (Dizin Ekle)
class AddIndex(model_name, index)[source]
model_name ile model için veritabanı tablosunda bir dizin oluşturur. index, Index sınıfının bir örneğidir.

RemoveIndex¶ (Dizini Kaldır)
class RemoveIndex(model_name, name)[source]
model_name modelinden name adlı indeksi kaldırır.

RenameIndex¶ (Dizini Yeniden Adlandır)
class RenameIndex(model_name, new_name, old_name=None, old_fields=None)[source]¶
model_name modelinin veritabanı tablosundaki bir indeksin adını değiştirir. old_name ve old_fields'tan sadece biri sağlanabilir. old_fields, genellikle index_together (Django 5.1 öncesi seçenek) alanlarına karşılık gelen, döngüsel bir dizidir.

Dizin yeniden adlandırma ifadesini desteklemeyen veritabanlarında (SQLite), işlem dizini siler ve yeniden oluşturur, bu da maliyetli olabilir.

AddConstraint¶ (Kısıtlama Ekle)
class AddConstraint(model_name, constraint)[source]
model_name modelinin veritabanı tablosunda bir kısıtlama oluşturur.

RemoveConstraint¶ (Kısıtlamayı Kaldır)
class RemoveConstraint(model_name, name)[source]
model_name modelinden name adlı kısıtlamayı kaldırır.

AlterConstraint¶ (Kısıtlamayı Değiştir)
Django 5.2'de yeni.
class AlterConstraint(model_name, name, constraint)[source]¶
Veritabanını etkilemeden model_name modelinin name adlı kısıtlamasını yeni kısıtlama ile değiştirir.

Special Operations (Özel İşlemler)¶
RunSQL¶
class RunSQL(sql, reverse_sql=None, state_operations=None, hints=None, elidable=False)[source]¶
Veritabanında rastgele SQL çalıştırmaya izin verir - Django'nun doğrudan desteklemediği veritabanı arka uçlarının daha gelişmiş özellikleri için kullanışlıdır.

sql ve varsa reverse_sql, veritabanında çalıştırılacak SQL dizeleri olmalıdır. Çoğu veritabanı arka ucunda (PostgreSQL hariç tümü), Django SQL'i yürütmeden önce tek tek ifadelere böler.

Uyarı

PostgreSQL ve SQLite'da, Django'nun işlem durumunu bozmamak için atomik olmayan geçişlerde SQL'inizde yalnızca BEGIN veya COMMIT kullanın.
Ayrıca bir dizi dizesi veya 2-tuple listesi de geçebilirsiniz. İkincisi, cursor.execute() ile aynı şekilde sorguları ve parametreleri geçmek için kullanılır. Bu üç işlem eşdeğerdir:

```
migrations.RunSQL("INSERT INTO musician (name) VALUES ('Reinhardt');")
migrations.RunSQL([("INSERT INTO musician (name) VALUES ('Reinhardt');", None)])
migrations.RunSQL([("INSERT INTO musician (name) VALUES (%s);", ["Reinhardt"])])
```

Sorguya gerçek yüzde işaretleri eklemek istiyorsanız, parametreleri aktarırken bunları iki katına çıkarmalısınız.

Reverse_sql sorguları, geçiş uygulanmadığında yürütülür. Sql sorguları tarafından yapılan işlemleri geri almalıdırlar. Örneğin, yukarıdaki eklemeyi silme işlemiyle geri almak için:

```
migrations.RunSQL(
```

```
    sql=[("INSERT INTO musician (name) VALUES (%s);", ["Reinhardt"])],
    reverse_sql=[("DELETE FROM musician where name=%s;", ["Reinhardt"])],
)
```

reverse_sql None ise (varsayılan), RunSQL işlemi geri alınamaz.

state_operations argümanı, proje durumu açısından SQL ile eşdeğer işlemler sağlamanıza olanak tanır. Örneğin, bir sütunu manuel olarak oluşturuyorsanız, otomatik algılayıcının modelin güncel durumuna sahip olmasını sağlamak için buraya AddField işlemini içeren bir liste geçmelisiniz. Bunu yapmazsanız, bir sonraki makemigrations çalıştırıldığında, bu alanı ekleyen herhangi bir işlem görmeyecek ve bu nedenle işlemi tekrar çalıştırmaya çalışacaktır. Örneğin:

```
migrations.RunSQL(
    "ALTER TABLE musician ADD COLUMN name varchar(255) NOT NULL;",
    state_operations=[
        migrations.AddField(
            "musician",
"name",
            models.CharField(max_length=255),
        ),
    ],
)
```

İsteğe bağlı hints argümanı, veritabanı yönlendiricilerinin yönlendirme kararlarını vermelerine yardımcı olmak için **hints olarak allow_migrate() yöntemine aktarılır. Veritabanı ipuçları hakkında daha fazla bilgi için İpuçları bölümüne bakın.

İsteğe bağlı elidable argümanı, göçleri sıkıştırırken işlemin kaldırılıp kaldırılmayacağını (elidable) belirler.

RunSQL.noop¶
İşlemin belirli bir yönde hiçbir şey yapmamasını istediğinizde RunSQL.noop özniteliğini sql veya reverse_sql'ye aktarın. Bu, işlemi geri alınabilir hale getirmek için özellikle yararlıdır.

RunPython¶
class RunPython(code, reverse_code=None, atomic=None, hints=None, elidable=False)[source]¶
Özel Python kodunu tarihsel bağlamda çalıştırır. code (ve varsa reverse_code) iki argüman kabul eden çağrılabilir nesneler olmalıdır; ilki, projenin geçmişindeki işlemin yerine uyan tarihsel modelleri içeren django.apps.registry.Apps örneğidir ve ikincisi SchemaEditor örneğidir.

reverse_code argümanı, göçleri geri alırken çağrılır. Bu çağrılabilir nesne, göçün geri alınabilir olması için code çağrılabilir nesnesinde yapılanları geri almalıdır. reverse_code None ise (varsayılan), RunPython işlemi geri alınamaz.

İsteğe bağlı hints argümanı, veritabanı yönlendiricilerinin yönlendirme kararını vermelerine yardımcı olmak için **hints olarak allow_migrate() yöntemine aktarılır. Veritabanı ipuçları hakkında daha fazla bilgi için İpuçları bölümüne bakın.

İsteğe bağlı elidable argümanı, göçleri sıkıştırırken işlemin kaldırılıp kaldırılmayacağını (elidable) belirler.

Kodu, göç dosyasında Migration sınıfının üzerinde ayrı bir işlev olarak yazmanız ve RunPython'a geçirmeniz önerilir. Aşağıda, RunPython'u kullanarak Country modelinde bazı başlangıç nesneleri oluşturmanın bir örneği verilmiştir:

```
from django.db import migrations


def forwards_func(apps, schema_editor):
    # We get the model from the versioned app registry;
    # İf we directly import it,it'll be the wrong version.
    Country = apps.get_model("myapp", "Country")
    db_alias = schema_editor.connection.alias
    Country.objects.using(db_alias).bulk_create(
        [
            Country(name="USA", code="us"),
            Country(name="France", code="fr"),
        ]
    )


def reverse_func(apps, schema_editor):
    # forwards_func ()creates two Country instances,
    # so reverse_func() should delete them.
    Country = apps.get_model("myapp", "Country")
    db_alias = schema_editor.connection.alias
    Country.objects.using(db_alias).filter(name="USA", code="us").delete()
    Country.objects.using(db_alias).filter(name="France", code="fr").delete()


class Migration(migrations.Migration):
    dependencies = []

    operations = [
        migrations.RunPython(forwards_func, reverse_func),
    ]
```

Bu genellikle veri göçlerini oluşturmak, özel veri güncellemelerini ve değişikliklerini çalıştırmak ve ORM ve/veya Python koduna erişmeniz gereken diğer her şeyi yapmak için kullanacağınız işlemdir.

RunSQL'e çok benzer şekilde, burada şemayı değiştirirseniz, bunu Django model sisteminin kapsamı dışında (ör. tetikleyiciler) yaptığınızdan veya SeparateDatabaseAndState kullanarak model durumundaki değişikliklerinizi yansıtacak işlemler eklediğinizden emin olun - aksi takdirde, sürümlenmiş ORM ve otomatik algılayıcı düzgün çalışmayı durdurur.

Varsayılan olarak, RunPython içeriğini DDL işlemlerini desteklemeyen veritabanlarında (örneğin, MySQL ve Oracle) bir işlem içinde çalıştırır. Bu güvenli olmalıdır, ancak bu arka uçlarda sağlanan schema_editor'ı kullanmaya çalışırsanız çökmeye neden olabilir; bu durumda, RunPython işlemine atomic=False değerini geçirin.

DDL işlemlerini destekleyen veritabanlarında (SQLite ve PostgreSQL), RunPython işlemlerine her geçiş için oluşturulan işlemlerin dışında otomatik olarak eklenen herhangi bir işlem yoktur. Bu nedenle, örneğin PostgreSQL'de, aynı geçişte şema değişikliklerini ve RunPython işlemlerini birleştirmekten kaçınmalısınız, aksi takdirde OperationalError: cannot ALTER TABLE "mytable" because it has pending trigger events gibi hatalarla karşılaşabilirsiniz.

Farklı bir veritabanınız varsa ve DDL işlemlerini destekleyip desteklemediğinden emin değilseniz, django.db.connection.features.can_rollback_ddl özniteliğini kontrol edin.

RunPython işlemi atomik olmayan bir göçün parçasıysa, işlem yalnızca atomic=True RunPython işlemine aktarılırsa bir işlemde yürütülür.

Uyarı

RunPython, modellerin bağlantısını sizin için sihirli bir şekilde değiştirmez; çağırdığınız tüm model yöntemleri, onlara geçerli veritabanı takma adını vermediğiniz sürece varsayılan veritabanına gider (bu takma ad, schema_editor.connection.alias'tan alınabilir; burada schema_editor, işlevinizin ikinci argümanıdır).

static RunPython.noop()[source]¶
İşlemin belirli bir yönde hiçbir şey yapmamasını istiyorsanız, RunPython.noop yöntemini code veya reverse_code'a geçirin. Bu, işlemi tersine çevrilebilir hale getirmek için özellikle yararlıdır.
SeparateDatabaseAndState¶(Ayrı Veritabanı Ve Eyalet)
class SeparateDatabaseAndState(database_operations=None, state_operations=None)[source]¶
İşlemlerin veritabanı (şema değiştirme) ve durum (otomatik algılayıcı güçlendirme) yönlerini karıştırıp eşleştirmenize olanak tanıyan oldukça özel bir işlemdir.

İki işlem listesi kabul eder. Durumu uygulaması istendiğinde, state_operations listesini kullanır (bu, RunSQL’nin state_operations argümanının genelleştirilmiş bir sürümüdür). Veritabanına değişiklikleri uygulaması istendiğinde, database_operations listesini kullanır.

Veritabanının gerçek durumu ile Django’nun durum görüşü senkronize olmazsa, bu durum geçiş çerçevesini bozabilir ve hatta veri kaybına yol açabilir. Dikkatli davranıp veritabanı ve durum işlemlerinizi dikkatlice kontrol etmeniz önemlidir. Veritabanı işlemlerinizi kontrol etmek için sqlmigrate ve dbshell kullanabilirsiniz. Durum işlemlerinizi kontrol etmek için makemigrations, özellikle --dry-run ile birlikte kullanabilirsiniz.

SeparateDatabaseAndState kullanımıyla ilgili bir örnek için, Bir through modeli kullanmak üzere ManyToManyField'ı değiştirme bölümüne bakın.
Operation category
class OperationCategory[source]
makemigrations komutu tarafından anlamlı semboller görüntülemek için kullanılan göç işlemi kategorileri.

ADDITION¶ (EKLEME)
Symbol: +

REMOVAL¶ (KALDIRMA)
Symbol: -

ALTERATION¶ (DEĞİŞİKLİK)
Symbol: ~

PYTHON¶
Symbol: p

SQL¶
Symbol: s

MIXED¶
Symbol: ?

Writing Your Own (Kendi işlemlnizi yazma) ¶
İşlemler nispeten basit bir API'ye sahiptir ve yerleşik Django işlemlerini tamamlamak için kendi işlemlerinizi kolayca yazabilmeniz için tasarlanmıştır. Bir İşlemin temel yapısı şöyledir:

```
from django.db.migrations.operations.base import Operation


class MyCustomOperation(Operation):
    # If this is False,it means that this operation will be ignored by
    # sqlmigrate;if true,it will be run and the SQL collected for its output.
    reduces_to_sql = False
```

```
    # If this isFalse,Django will refuse to reverse past this operation.
    reversible = False

    # This categorizes the operation. The corresponding symbol will be
# displayed by the makemigrations command.
    category = OperationCategory.ADDITION

    def __init__(self, arg1, arg2):
        # Operations are usually instantiated with arguments in migration.
        # files. Store the values of them on self for later use.
        pass

    def state_forwards(self, app_label, state):
        # The Operation should take the 'state' parameter (an instance of
#
django.db.migrations.state.ProjectState) and mutate it to match
# any schema changes that have occurred.
        pass

    def database_forwards(self, app_label, schema_editor, from_state, to_state):
        # The Operation should use schema_editor to apply any changes it
# wants to make to the database.
        pass

    def database_backwards(self, app_label, schema_editor, from_state, to_state):
        # If reversible is True, this is called when the operation is reversed.
        pass
def describe(self):
        # This is used to describe what the operation does.
        return "Custom Operation "

    @property
    def migration_name_fragment(self):
        # Optional. A filename part suitable for automatically naming a
#migration containing this operation, or None if not applicable.
        return "custom_operation_%s_%s" % (self.arg1, self.arg2)
```

Bu şablonu alıp üzerinde çalışabilirsiniz, ancak django.db.migrations.operations içindeki yerleşik Django işlemlerine bakmanızı öneririz. Bu işlemler, ProjectState ve geçmiş modelleri almak için kullanılan kalıplar gibi göç çerçevesinin yarı-dahili yönlerinin birçok örnek kullanımını ve ModelState ile state_forwards() içinde geçmiş modelleri değiştirmek için kullanılan kalıpları kapsar.

Dikkat edilmesi gereken bazı noktalar:

Göçleri yazmak için ProjectState hakkında çok fazla şey öğrenmenize gerek yoktur; sadece, bir uygulama kayıt defterine erişim sağlayan bir apps özelliği olduğunu bilmeniz yeterlidir (burada get_model işlevini çağırabilirsiniz).

database_forwards ve database_backwards, her ikisi de kendilerine aktarılan iki durum alır; bunlar, state_forwards yönteminin uygulayacağı farkı temsil eder, ancak kolaylık ve hız nedenleriyle size verilir.

database_forwards() veya database_backwards() içindeki from_state argümanından model sınıfları veya model örnekleri ile çalışmak istiyorsanız, ilgili modelleri kullanılabilir hale getirmek için clear_delayed_apps_cache() yöntemini kullanarak model durumlarını oluşturmalısınız:

```
def database_forwards(self, app_label, schema_editor, from_state, to_state):
    # Bu işlem tüm modellere erişebilmelidir. Herhangi bir gecikme olması durumunda
tüm modellerin
    # yeniden yüklendiğinden emin olun.
    from_state.clear_delayed_apps_cache()
    ...
```

database_backwards yöntemindeki to_state, eski durumdur; yani, geçişin tersine çevrilmesi tamamlandığında geçerli durum olacak olan durumdur.

Yerleşik işlemlerde references_model uygulamaları görebilirsiniz; bu, otomatik algılama kodunun bir parçasıdır ve özel işlemler için önemli değildir.

Uyarı

Performans nedenleriyle, ModelState.fields içindeki Field örnekleri göçler arasında yeniden kullanılır. Bu örneklerdeki öznitelikleri asla değiştirmemelisiniz. state_forwards() içinde bir alanı değiştirmek gerekirse, eski örneği ModelState.fields içinden kaldırmalı ve yerine yeni bir örnek eklemelisiniz. Aynısı ModelState.managers içindeki Manager örnekleri için de geçerlidir.

Örnek olarak, PostgreSQL uzantılarını (PostgreSQL'in daha heyecan verici özelliklerinden bazılarını içeren) yükleyen bir işlem yapalım. Model durumunda herhangi bir değişiklik olmadığı için, tek yaptığı tek bir komutu çalıştırmaktır:

```
from django.db.migrations.operations.base import Operation


class LoadExtension(Operation):
    reversible = True

    def __init__(self, name):
```

```
        self.name = name

    def state_forwards(self, app_label, state):
        pass

    def database_forwards(self, app_label, schema_editor, from_state, to_state):
        schema_editor.execute("CREATE EXTENSION IF NOT EXISTS %s" % self.name)

    def database_backwards(self, app_label, schema_editor, from_state, to_state):
        schema_editor.execute("DROP EXTENSION %s" % self.name)

    def describe(self):
        return "Creates extension %s" % self.name

    @property
    def migration_name_fragment(self):
        return "create_extension_%s" % self.name
```

Erbil Bayçoral 2024481015

SchemaEditor¶
class BaseDatabaseSchemaEditor[source]¶
Django’nun geçiş sistemi iki bölüme ayrılmıştır: hangi işlemlerin çalıştırılması gerektiğini hesaplama ve depolama mantığı (django.db.migrations) ve “model oluşturma” veya “alan silme” gibi işlemleri SQL’e dönüştüren veritabanı soyutlama katmanı - bu SchemaEditor’ın görevidir
.Django kullanan normal bir geliştirici olarak SchemaEditor ile doğrudan etkileşim kurmak istemezsiniz, ancak kendi migration sisteminizi yazmak istiyorsanız veya daha gelişmiş ihtiyaçlarınız varsa, SQL yazmaktan çok daha iyidir. Django'daki her veritabanı arka ucu kendi SchemaEditor sürümünü sağlar ve her zaman connection.schema_editor() kontekst yöneticisi aracılığıyla erişilebilir:

```
with connection.schema_editor() as schema_editor:
    schema_editor.delete_model(MyModel)
```

İşlemleri ve ertelenmiş SQL'leri (ForeignKey kısıtlamaları oluşturmak gibi) yönetebilmesi için konteks yöneticisi aracılığıyla kullanılmalıdır.
Tüm olası işlemleri, değişikliklerin uygulanmasını istediğiniz sırayla çağrılması gereken yöntemler olarak ortaya çıkarır. Bazı olası işlemler veya değişiklik türleri tüm veritabanlarında mümkün değildir - örneğin, MyISAM yabancı anahtar kısıtlamalarını desteklemez.
Django için üçüncü taraf bir veritabanı arka ucu yazıyor veya bakımını yapıyorsanız, Django'nun geçiş işlevselliği ile çalışmak için bir SchemaEditor uygulaması sağlamanız gerekir - ancak, veritabanınız SQL ve ilişkisel veritabanı kullanımında nispeten standart olduğu sürece yerleşik , Django SchemaEditor sınıflarından birinin alt sınıfını oluşturabilir ve sözdizimini biraz değiştirebilirsiniz.

Yöntemler¶

execute()¶
BaseDatabaseSchemaEditor.execute(sql, params=())[source]¶
Verilen parametrelerle birlikte, iletilen SQL ifadesini yürütür. Bu, normal veritabanı imleçlerini saran bir sarmalayıcıdır ve kullanıcı isterse SQL'i bir .sql dosyasına kaydetmeye olanak tanır.
create_model()¶
BaseDatabaseSchemaEditor.create_model(model)[source]¶
Verilen model için veritabanında yeni bir tablo oluşturur, bununla birlikte gerekli olan benzersiz kısıtlamaları veya dizinleri de oluşturur.

delete_model()¶
BaseDatabaseSchemaEditor.delete_model(model)[source]¶
Veritabanındaki modelin tablosunu, sahip olduğu benzersiz kısıtlamalar veya dizinlerle birlikte siler.

add_index()¶
BaseDatabaseSchemaEditor.add_index(model, index)[source]¶
Modelin tablosuna dizin ekler.

remove_index()¶
BaseDatabaseSchemaEditor.remove_index(model, index)[source]¶
Modelin tablosundan dizini kaldırır.

rename_index()¶
BaseDatabaseSchemaEditor.rename_index(model, old_index, new_index)[source]¶
Model tablosundaki eski_dizin adını yeni_dizin olarak değiştirir.

add_constraint()¶
BaseDatabaseSchemaEditor.add_constraint(model, constraint)[source]¶
Model tablosuna kısıtlama ekler.

remove_constraint()¶
BaseDatabaseSchemaEditor.remove_constraint(model, constraint)[source]¶
Modelin tablosundan kısıtlamayı kaldırır.

alter_unique_together()¶
BaseDatabaseSchemaEditor.alter_unique_together(model, old_unique_together, new_unique_together)[source¶
Modelin unique_together değerini değiştirir; bu, yeni değerle eşleşene kadar modelin tablosundan benzersiz kısıtlamaları ekler veya kaldırır.

alter_index_together()¶
BaseDatabaseSchemaEditor.alter_index_together(model, old_index_together, new_index_together)[source]¶
Bir modelin index_together değerini değiştirir; bu, yeni değerle eşleşene kadar modelin tablosuna indeksler ekler veya çıkarır.

alter_db_table()¶
BaseDatabaseSchemaEditor.alter_db_table(model, old_db_table, new_db_table)[source]¶
Modelin tablosunun adını old_db_table'dan new_db_table'a değiştirir.

alter_db_table_comment()¶
BaseDatabaseSchemaEditor.alter_db_table_comment(model, old_db_table_comment, new_db_table_comment)[source]¶
Modelin tablo yorumunu new_db_table_comment olarak değiştirir.

alter_db_tablespace()¶
BaseDatabaseSchemaEditor.alter_db_tablespace(model, old db_tablespace, new_db_tablespace)[source]¶
Modelin tablosunu bir tablo alanından başka bir tablo alanına taşır.

add_field()¶
BaseDatabaseSchemaEditor.add_field(model, field)[source]¶
Alanı temsil etmek için modelin tablosuna bir sütun (veya bazen birden fazla sütun) ekler. Bu, alanın db_index=True veya unique=True olması durumunda dizinler veya benzersiz bir kısıtlama da ekler.

Alan, through değeri olmayan bir ManyToManyField ise, sütun oluşturmak yerine, ilişkiyi temsil eden bir tablo oluşturur. through değeri sağlanırsa, bu işlem yapılmaz.

Alan bir ForeignKey ise, bu aynı zamanda sütuna yabancı anahtar kısıtlaması da ekler.

remove_field()¶
BaseDatabaseSchemaEditor.remove_field(model, field)[source]¶
Modelin tablosundan alanı temsil eden sütunları, bu alanın neden olduğu tüm benzersiz kısıtlamalar, yabancı anahtar kısıtlamaları veya dizinlerle birlikte kaldırır. Alan, through değeri olmayan bir ManyToManyField ise, ilişkiyi izlemek için oluşturulan tabloyu kaldırır. through değeri sağlanırsa, hiçbir işlem yapılmaz.

alter_field()¶
BaseDatabaseSchemaEditor.alter_field(model, old_field, new_field, strict=False)[source]¶
Bu, modeldeki alanı eski alandan yeni alana dönüştürür. Bu, sütunun adını (db_column özniteliği) değiştirmeyi, alanın türünü değiştirmeyi (alan sınıfı değişirse), alanın NULL durumunu değiştirmeyi, yalnızca alana özgü benzersiz kısıtlamalar ve dizinler eklemeyi veya kaldırmayı, birincil anahtarı değiştirmeyi ve ForeignKey kısıtlamalarının hedefini değiştirmeyi içerir.

Bu işlemin yapamayacağı en yaygın dönüşüm, ManyToManyField'ı normal bir Field'a veya tersini dönüştürmektir; Django bunu veri kaybı olmadan yapamaz ve bu nedenle bunu yapmayı reddeder. Bunun yerine, remove_field() ve add_field() ayrı ayrı çağrılmalıdır.

Veritabanında supports_combined_alters varsa, Django bunları tek bir veritabanı çağrısında mümkün olduğunca çok yapmaya çalışır; aksi takdirde, her değişiklik için ayrı bir ALTER ifadesi verir, ancak değişiklik gerekmeyen yerlerde ALTER ifadesi vermez.

Attributes¶
Aksi belirtilmedikçe, tüm özellikler salt okunur olarak kabul edilmelidir.

connection¶
SchemaEditor.connection¶
Veritabanına bir bağlantı nesnesi. Bağlantının yararlı bir özelliği, erişilen veritabanının adını belirlemek için kullanılabilen takma addır.

Bu, birden fazla veritabanı içeren migrations için veri migrations yaparken yararlıdır.

How to create database migrations¶
Bu belge, karşılaşabileceğiniz farklı senaryolar için veritabanı geçişleri nasıl yapılandırılacağını ve yazılacağını açıklamaktadır. Geçişler hakkında giriş niteliğinde bilgi için konu kılavuzuna bakın.
Data migrations and multiple databases¶
Birden çok veritabanı kullanırken, belirli bir veritabanına geçiş yapıp yapmama kararını vermeniz gerekebilir. Örneğin, yalnızca belirli bir veritabanında geçiş yapmak isteyebilirsiniz.
Bunu yapmak için, schema_editor.connection.alias özniteliğine bakarak RunPython işlemi içindeki veritabanı bağlantısının takma adını kontrol edebilirsiniz:

```
from django.db import migrations
def forwards(apps, schema_editor):
    if schema_editor.connection.alias != "default":
return
    # Your migration code goes here
class Migration(migrations.Migration):
    dependencies = [
        # Dependencies to other migrations
    ]
    operations = [
        migrations.RunPython(forwards),
    ]
```

Ayrıca, **hints olarak veritabanı yönlendiricilerinin allow_migrate() yöntemine aktarılacak ipuçları da sağlayabilirsiniz:
myapp/dbrouters.py¶

```
class MyRouter:
    def allow_migrate(self, db, app_label, model_name=None, **hints):
        if "target_db" in hints:
            return db == hints["target_db"]
        return True
```

Ardından, bunu migration'larınızda kullanmak için aşağıdakileri yapın:

```
from django.db import migrations
```

```
def forwards(apps, schema_editor):
    # Your migration code goes here
    ...
class Migration(migrations.Migration):
    dependencies = [
        # Dependencies to other migrations
    ]
    operations = [
        migrations.RunPython(forwards, hints={"target_db": "default"}),    ]
```

RunPython veya RunSQL işleminiz yalnızca bir modeli etkiliyorsa, yönlendiriciye mümkün olduğunca şeffaf hale getirmek için model_name'i ipucu olarak iletmek iyi bir uygulamadır. Bu, yeniden kullanılabilir ve üçüncü taraf uygulamalar için özellikle önemlidir.

Migrations that add unique fields¶

Mevcut satırları olan bir tabloya benzersiz, null olamayan bir alan ekleyen “düz” bir geçiş uygulamak, mevcut satırları doldurmak için kullanılan değer yalnızca bir kez üretildiğinden, benzersiz kısıtlamayı bozarak bir hata oluşturacaktır.

Bu nedenle, aşağıdaki adımlar atılmalıdır. Bu örnekte, varsayılan değeri olan null olamayan bir UUIDField ekleyeceğiz. İlgili alanı ihtiyaçlarınıza göre değiştirin.

default=uuid.uuid4 ve unique=True argümanlarıyla modelinize alanı ekleyin (eklediğiniz alanın türüne uygun bir varsayılan değer seçin).

makemigrations komutunu çalıştırın. Bu, AddField işlemiyle bir geçiş oluşturmalıdır.

makemigrations myapp --empty komutunu iki kez çalıştırarak aynı uygulama için iki boş geçiş dosyası oluşturun. Aşağıdaki örneklerde, geçiş dosyalarına anlamlı isimler vermek için isimlerini değiştirdik.

Otomatik olarak oluşturulan geçişten (üç yeni dosyanın ilki) AddField işlemini son geçişe kopyalayın, AddField'ı AlterField olarak değiştirin ve uuid ve models içe aktarmalarını ekleyin. Örneğin:

0006_remove_uuid_null.py¶

```
# Generated by Django A.B on YYYY-MM-DD HH:MM
from django.db import migrations, models
import uuid
class Migration(migrations.Migration):
    dependencies = [
        ("myapp", "0005_populate_uuid_values"),
    ]
    operations = [
        migrations.AlterField(
            model_name="mymodel",
            name="uuid",
            field=models.UUIDField(default=uuid.uuid4, unique=True),
        ),
    ]
```

İlk migration dosyasını düzenleyin. Oluşturulan migration sınıfı şuna benzer olmalıdır:

0004_add_uuid_field.py¶

```
class Migration(migrations.Migration):
    dependencies = [
        ("myapp", "0003_auto_20150129_1705"),
    ]    operations = [      migrations.AddField(  model_name="mymodel",
name="uuid",           field=models.UUIDField(default=uuid.uuid4,
unique=True),    ),    ]
```

unique=True değerini null=True olarak değiştirin – bu, ara null alanı oluşturacak ve tüm satırlara benzersiz değerler girilene kadar benzersiz kısıtlamanın oluşturulmasını erteleyecektir.

İlk boş geçiş dosyasına, mevcut her satır için benzersiz bir değer (örnekte UUID) oluşturmak üzere bir RunPython veya RunSQL işlemi ekleyin. Ayrıca uuid içe aktarımını da ekleyin. Örneğin:

0005_populate_uuid_values.py¶

```
# Generated by Django A.B on YYYY-MM-DD HH:MM
from django.db import migrations
import uuid
def gen_uuid(apps, schema_editor):
    MyModel = apps.get_model("myapp", "MyModel")
    for row in MyModel.objects.all():
        row.uuid = uuid.uuid4()
        row.save(update_fields=["uuid"])
class Migration(migrations.Migration):
    dependencies = [
        ("myapp", "0004_add_uuid_field"),
    ]
    operations = [
        # omit reverse_code=... if you don't want the migration to be reversible.
        migrations.RunPython(gen_uuid, reverse_code=migrations.RunPython.noop),
    ]
```

Artık migrate komutuyla her zamanki gibi migration'ları uygulayabilirsiniz.

Bu migration çalışırken nesnelerin oluşturulmasına izin verirseniz bir yarışma durumu oluşur. AddField'dan sonra ve RunPython'dan önce oluşturulan nesnelerin orijinal uuid'leri üzerine yazılacaktır.

Non-atomic migrations¶

DDL işlemlerini destekleyen veritabanlarında (SQLite ve PostgreSQL), göçler varsayılan olarak bir işlem içinde çalışır. Büyük tablolarda veri migrations gerçekleştirmek gibi kullanım durumları için, atomik özniteliğini False olarak ayarlayarak bir göçün işlem içinde çalışmasını engelleyebilirsiniz:

```
from django.db import migrations
class Migration(migrations.Migration):
    atomic = False
```

Böyle bir geçişte, tüm işlemler işlem olmadan yürütülür. atomic() kullanarak veya RunPython'a atomic=True aktararak geçişin bir kısmını işlem içinde yürütmek mümkündür.

Aşağıda, büyük bir tabloyu daha küçük gruplar halinde güncelleyen atomik olmayan bir veri geçişi örneği verilmiştir:

```
import uuid
```

```
from django.db import migrations, transaction
def gen_uuid(apps, schema_editor):
    MyModel = apps.get_model("myapp", "MyModel")
    while MyModel.objects.filter(uuid__isnull=True).exists():
        with transaction.atomic():
            for row in MyModel.objects.filter(uuid__isnull=True)[:1000]:
                row.uuid = uuid.uuid4()
                row.save()
class Migration(migrations.Migration):
    atomic = False
    operations = [
        migrations.RunPython(gen_uuid),
    ]
```

atomic özniteliği, DDL işlemlerini desteklemeyen veritabanlarında (ör. MySQL, Oracle) etkili değildir. (MySQL'in atomic DDL deyimi desteği, geri alınabilen bir işlemde birden fazla deyimi değil, tek tek deyimleri ifade eder.)

Controlling the order of migrations¶

Django, göçlerin uygulanacağı sırayı her bir göçün dosya adına göre değil, Migration sınıfındaki iki özellik olan dependencies ve run_before kullanarak bir grafik oluşturarak belirler.

makemigrations komutunu kullandıysanız, otomatik olarak oluşturulan göçlerde bu özellik oluşturma sürecinin bir parçası olarak tanımlandığından, dependencies özelliğinin nasıl çalıştığını muhtemelen görmüşsünüzdür.

Bağımlılıklar özelliği şu şekilde tanımlanır:

```
from django.db import migrations
class Migration(migrations.Migration):
    dependencies = [
        ("myapp", "0123_the_previous_migration"),
    ]
```

Genellikle bu yeterli olacaktır, ancak zaman zaman geçişinizin diğer geçişlerden önce çalışmasını sağlamanız gerekebilir. Bu, örneğin, üçüncü taraf uygulamalarının geçişlerinin AUTH_USER_MODEL değiştirmenizin ardından çalışmasını sağlamak için yararlıdır.

Bunu başarmak için, sizinkine bağlı olması gereken tüm geçişleri Migration sınıfınızın run_before özniteliğine yerleştirin:

```
class Migration(migrations.Migration):
    ...
    run_before = [
        ("third_party_app", "0001_do_awesome"),
    ]
```

Mümkün olduğunda run_before yerine bağımlılıkları kullanmayı tercih edin. Yalnızca, yazdığınız göçün ardından çalıştırmak istediğiniz göçte bağımlılıkları belirtmenin istenmeyen veya pratik olmadığı durumlarda run_before kullanmalısınız.

Migrating data between third-party apps¶

Veri göçünü kullanarak bir üçüncü taraf uygulamasından başka bir uygulamaya veri taşıyabilirsiniz.

Eski uygulamayı daha sonra kaldırmayı planlıyorsanız, eski uygulamanın kurulu olup olmadığına göre bağımlılıklar özelliğini ayarlamanız gerekir. Aksi takdirde, eski uygulamayı kaldırdığınızda bağımlılıklar eksik kalır. Benzer şekilde, eski uygulamadan modelleri alan apps.get_model() çağrısında LookupError hatasını yakalamanız gerekir. Bu yaklaşım, eski uygulamayı önce yükleyip sonra kaldırmadan projenizi herhangi bir yere dağıtmanıza olanak tanır.
İşte bir örnek geçiş:
myapp/migrations/0124_move_old_app_to_new_app.py¶

```
from django.apps import apps as global_apps
from django.db import migrations
def forwards(apps, schema_editor):
    try:
        OldModel = apps.get_model("old_app", "OldModel")
    except LookupError:
        # The old app isn't installed.
        return
    NewModel = apps.get_model("new_app", "NewModel")
    NewModel.objects.bulk_create(
        NewModel(new_attribute=old_object.old_attribute)
        for old_object in OldModel.objects.all()    )class Migration(migrations.Migration):
    operations = [
        migrations.RunPython(forwards, migrations.RunPython.noop),
    ]
    dependencies = [
        ("myapp", "0123_the_previous_migration"),
        ("new_app", "0001_initial"),
    ]
    if global_apps.is_installed("old_app"):
        dependencies.append(("old_app", "0001_initial"))
```

Ayrıca, geçişin uygulanmaması durumunda ne olmasını istediğinizi de düşünün. Hiçbir şey yapmayabilir (yukarıdaki örnekte olduğu gibi) veya yeni uygulamadan verilerin bir kısmını veya tamamını silebilirsiniz. RunPython işleminin ikinci argümanını buna göre ayarlayın.
Changing a ManyToManyField to use a through model¶
ManyToManyField'ı bir geçiş modeli kullanacak şekilde değiştirirseniz, varsayılan geçiş mevcut tabloyu siler ve yeni bir tablo oluşturur, böylece mevcut ilişkiler kaybolur. Bunu önlemek için SeparateDatabaseAndState kullanarak mevcut tablonun adını yeni tablo adıyla değiştirebilir ve geçiş otomatik algılayıcısına yeni modelin oluşturulduğunu bildirebilirsiniz. Mevcut tablo adını ve kısıtlama adını sqlmigrate veya dbshell aracılığıyla kontrol edebilirsiniz. Yeni tablo adını, through modelinin _meta.db_table özelliği ile kontrol edebilirsiniz. Yeni through modeliniz, Django'nun yaptığı gibi ForeignKeys için aynı adları kullanmalıdır. Ayrıca, ekstra alanlara ihtiyaç duyarsa, bunlar SeparateDatabaseAndState işleminden sonra eklenmelidir.
Örneğin, Author'a bağlanan ManyToManyField içeren bir Book modelimiz varsa, yeni bir is_primary alanı içeren bir through modeli AuthorBook ekleyebiliriz, şöyle:

```
from django.db import migrations, models
import django.db.models.deletion
```

```
class Migration(migrations.Migration):
    dependencies = [
        ("core", "0001_initial"),
    ]
    operations = [
        migrations.SeparateDatabaseAndState(
            database_operations=[
                # Old table name from checking with sqlmigrate, new table
                # name from AuthorBook._meta.db_table.
                migrations.RunSQL(
                    sql="ALTER TABLE core_book_authors RENAME TO core_authorbook",
                    reverse_sql="ALTER TABLE core_authorbook RENAME TO
core_book_authors",
                ),
            ],
            state_operations=[
                migrations.CreateModel(
                    name="AuthorBook",
                    fields=[
                        (
                            "id",
                            models.AutoField(
                                auto_created=True,
                                primary_key=True,
                                serialize=False,
                                verbose_name="ID",
                            ),
                        ),
                        (
                            "author",
                            models.ForeignKey(
                                on_delete=django.db.models.deletion.DO_NOTHING,
                                to="core.Author",
                            ),
                        ),
                        (
                            "book",
                            models.ForeignKey(
                                on_delete=django.db.models.deletion.DO_NOTHING,
                                to="core.Book",
                            ),
                        ),                    ],
                    options={
                        "constraints": [
                            models.UniqueConstraint(
                                fields=["author", "book"],
                                name="unique_author_book",
```

```
                )
            ],
        },
    ),
    migrations.AlterField(
        model_name="book",
        name="authors",
        field=models.ManyToManyField(
            to="core.Author",
            through="core.AuthorBook",
        ),
    ),
  ],
),
migrations.AddField(
  model_name="authorbook",
  name="is_primary",
  field=models.BooleanField(default=False),
),
]
```

Changing an unmanaged model to managed¶

Yönetilmeyen bir modeli (managed=False) yönetilene dönüştürmek istiyorsanız, modele şema ile ilgili diğer değişiklikleri yapmadan önce managed=False ifadesini kaldırmanız ve bir geçiş oluşturmanız gerekir. Çünkü Meta.managed öğesini değiştirme işlemini içeren geçişte görünen şema değişiklikleri uygulanmayabilir.